

Estudiante: Karla Ospina Fuentes

1. Descripción del Ejercicio

En este taller implementé un sistema de figuras geométricas aplicando los principios de la Programación Orientada a Objetos en Python. El objetivo fue demostrar el uso de encapsulamiento mediante atributos privados con `@property` y `@setter`, herencia en una jerarquía de clases, sobrescritura de métodos, validaciones internas y el estándar PEP8.

Los conceptos aplicados fueron:

- Encapsulamiento: atributos privados `_ancho` y `_alto` con `@property` y `@setter`
- Herencia: tres clases hijas que heredan de `FiguraGeometrica`
- Sobrescritura de métodos: `area()`, `perimetro()` y `__str__()` en cada clase
- Validaciones: se lanza `ValueError` si los valores son menores o iguales a 0
- Estándar PEP8: nombres en `snake_case`, comentarios, espaciado correcto

2. Estructura de Clases

FiguraGeometrica es la clase base del sistema. En ella definí dos atributos privados, `_ancho` y `_alto`, protegidos mediante `@property` para la lectura y `@setter` para la escritura. Agregué validaciones que lanzan un `ValueError` si algún valor es menor o igual a cero. También implementé el método `area()` con la fórmula general ancho por alto, y dejé `perimetro()` con `pass` para que cada clase hija lo sobrescriba con su propia fórmula.

Cuadrado hereda de `FiguraGeometrica` y recibe un único parámetro llamado `lado`. En el constructor llamé a `super().__init__(ancho=lado, alto=lado)` para asignar el mismo valor a ambas dimensiones. Sobreescríbí `area()` como `lado` al cuadrado y `perimetro()` como 4 multiplicado por el lado.

Rectangulo también hereda de `FiguraGeometrica` y recibe ancho y alto de forma independiente. Sobreescríbí `area()` como ancho por alto, y `perimetro()` como 2 multiplicado por la suma de ancho más alto.

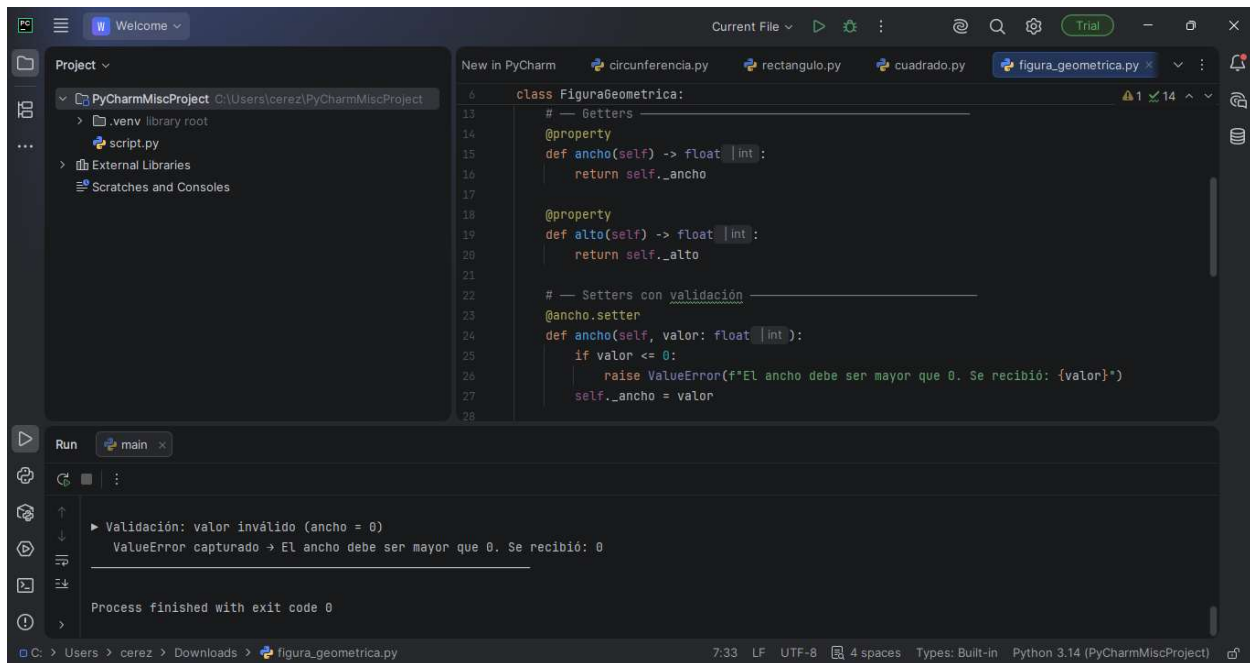
Circunferencia hereda de `FiguraGeometrica` y recibe únicamente el radio, que según el diagrama del taller se asigna al atributo ancho. Sobreescríbí `area()` con la fórmula π por radio al cuadrado, y `perimetro()` con 2 por π por radio, usando el módulo `math` de Python para mayor precisión.

3. Explicación de Cada Clase

3.1 FiguraGeometrica (figura_geometrica.py)

Es la clase base del sistema. Definí los atributos privados `_ancho` y `_alto`, protegiéndolos con `@property` para la lectura y `@setter` para la escritura con validación. Si alguien intenta asignar un valor menor o igual a cero, el programa lanza un `ValueError` de forma automática. También implementé el método `area()` que calcula ancho por alto, y dejé `perimetro()` con `pass` para que cada clase hija lo sobrescriba según su fórmula.

Ejemplo del setter con validación:



3.2 Cuadrado (cuadrado.py)

Esta clase hereda de `FiguraGeometrica` y recibe un único parámetro llamado `lado`. Llamé a `super().__init__(ancho=lado, alto=lado)` para asignar el mismo valor a ambas dimensiones, ya que en un cuadrado todos los lados son iguales. Sobreescribí `area()` como `lado` al cuadrado y `perimetro()` como `4` por `lado`.

3.3 Rectangulo (rectangulo.py)

Hereda de `FiguraGeometrica` y recibe `ancho` y `alto` de forma independiente. Sobreescribí `area()` como `ancho` por `alto`, y `perimetro()` como `2` por la suma de `ancho` más `alto`. Es la implementación más directa de la clase base.

3.4 Circunferencia (circunferencia.py)

Hereda de `FiguraGeometrica` y recibe solo el `radio`. Según el diagrama del taller, el `radio` se asigna al `ancho`. Sobreescribí `area()` con la fórmula π por `radio` al cuadrado, y `perimetro()` con 2 por π por `radio`. Utilicé el módulo `math` de Python para usar π con precisión.

3.5 main.py

Es el programa principal donde creé instancias de cada figura, llamé a sus métodos y verifiqué la herencia con `isinstance()`. También probé las validaciones pasando valores negativos y cero para confirmar que el `ValueError` se lanza correctamente.

4. Resultados de la Ejecución

```
TALLER: Figuras Geométricas con P00

► Cuadrado (lado = 5)
Cuadrado      | Lado: 5 | Área: 25.00 | Perímetro: 20.00

► Rectángulo (ancho = 8, alto = 4)
Rectángulo    | Ancho: 8 | Alto: 4 | Área: 32.00 | Perímetro: 24.00

► Circunferencia (radio = 7)
Circunferencia | Radio: 7 | Área: 153.94 | Perímetro: 43.98

► Verificación de herencia con isinstance():
Cuadrado      es FiguraGeometrica: True
Rectángulo    es FiguraGeometrica: True
Circunferencia es FiguraGeometrica: True
```

```
Circunferencia | Radio: 7 | Área: 153.94 | Perímetro: 43.98

► Verificación de herencia con isinstance():
Cuadrado      es FiguraGeometrica: True
Rectángulo    es FiguraGeometrica: True
Circunferencia es FiguraGeometrica: True

► Validación: valor inválido (lado = -3)
ValueError capturado → El ancho debe ser mayor que 0. Se recibió: -3

► Validación: valor inválido (ancho = 0)
ValueError capturado → El ancho debe ser mayor que 0. Se recibió: 0

Process finished with exit code 0
```

5. Conclusión

Con este taller logré aplicar correctamente los pilares de POO en Python. El encapsulamiento me permitió proteger los datos internos de cada figura y garantizar que siempre sean válidos. La herencia evitó repetir código al compartir la estructura base entre Cuadrado, Rectangulo y Circunferencia. La sobrescritura de métodos permitió que cada figura calcule su propia área y perímetro con la fórmula correcta, manteniendo una interfaz común y consistente.